

Four Degrees of Freedom Robotic Arm Design and Implementation

*Furkan Güntaş, Orhan Köseoğlu, Emircan Yücel
Kadir Has University*

Abstract— This project focuses on the design and implementation of a 4-degree-of-freedom (DOF) robotic arm for pick-and-place tasks. The robot has a lightweight design and uses Denavit-Hartenberg parameters for precise kinematic modeling. The CAD model includes 3D design details of the robotic arm. The design combines forward and inverse kinematics, dynamic modeling, actuator selection, and PID control to ensure stable operation. The system uses PID algorithms implemented in SIMULINK and trajectory simulations performed in MATLAB for precise motion control. Safety and efficiency are prioritized to ensure reliable manipulation within the defined working area.

Index Terms— Denavit-Hartenberg, Jacobian Matrix, Forward Kinematics, Manipulator Dynamic, Inverse Kinematics, PID Control, MATLAB, SIMULINK.

I. INTRODUCTION

The primary objective of this project is to design and implement a 4-degree-of-freedom (DOF) robotic arm for performing pick-and-place operations, focusing on both practical application and theoretical foundation. Central to the project is the development of a comprehensive kinematic and dynamic model of the robotic system. Utilizing Denavit-Hartenberg (DH) parameters, the forward and inverse kinematics are derived to ensure precise end-effector positioning. The dynamic analysis incorporates critical components such as the inertia matrix, Coriolis forces, and gravitational effects, enabling the accurate calculation of joint torques and accelerations. These calculations were fundamental in guiding actuator and microcontroller selection to ensure optimal performance and reliability. The control system employs a PID algorithm designed in the SIMULINK environment to stabilize the system and mitigate disturbances during operation. Custom trajectory planning was carried out in MATLAB using high-order polynomials to ensure smooth and efficient motion across all joints, with detailed evaluations of angular positions, velocities, and accelerations. This project adopts a modular and user-centered approach, allowing future customization in DOF and parameter definitions. The methodology integrates CAD modeling, control system design, and dynamic analysis to deliver a versatile and high-performing robotic arm. By combining simulation and practical implementation, the project aims to bridge the gap between theoretical concepts and real-world robotic applications, providing valuable insights for advanced robotics design and control.

II. GENERAL SYSTEM DEFINITIONS AND DETAILS

A. Task Requirements

The manipulator designed for this project has an RRRR

configuration, where all joints are revolute. It has a simple gripper-like end effector designed to pick up objects from one location and place them in another. The manipulator moves in a three-dimensional space with four degrees of freedom. This configuration allows the robot to lift and carry objects autonomously.

Its main function is to improve efficiency and speed in industrial applications such as assembly lines or material handling processes. For example, in an automotive assembly line, parts like hoods or fenders are positioned autonomously and mounted onto vehicles with high precision. Similar systems are also widely used in industries such as furniture manufacturing, aerospace production, and other industrial fields.



Figure 1 Example of 4-DOF RRRR robotic arm.

B. CAD Model

Every component of the manipulator system, including the gripper-like end effector, is fully represented in the CAD model (See Figure 2). The manipulator's RRRR configuration is reflected in the model, with revolute joints enabling precise movements. SolidWorks was chosen as the CAD program due to its advanced capabilities and its suitability for designing complex robotic systems in this project.



Figure 2 CAD design in Solidworks.

C. Degrees of Freedom

The manipulator's four degrees of freedom (DoFs) enable it to move freely along the X, Y, and Z axes, providing comprehensive motion capabilities in three-dimensional space. In addition to translational movement, the manipulator's rotational ability around the Z axis allows it to perform tasks such as box organization with greater flexibility and precision. The rotation of the end effector is achieved through revolute joints, which play a crucial role in enhancing the manipulator's overall functionality. These joints enable the system to perform rotational movements that are essential for tasks requiring specific orientations, such as arranging boxes in preferred configurations.

Having more than three degrees of freedom offers significant advantages. It enhances the manipulator's flexibility and efficiency, enabling it to adapt to a wide variety of tasks. This added capability allows for unique and optimized approaches to box organization, making the manipulator a versatile and effective tool in various applications.

D. Materials Selection and Production

When choosing the material for the robot's links, consideration was given to variables such as maximum torque, overall size, and the expected load, represented by the tasks the manipulator is designed to perform. Density, Young's modulus, and thermal expansion coefficient are key mechanical properties of the selected materials, ensuring optimal performance in terms of precision and durability. These factors are crucial to maintaining the manipulator's functionality across diverse operational conditions.

For the prototype phase of this project, PLA (polylactic acid) material was chosen due to its cost-effectiveness, ease of manufacturing, and compatibility with 3D printing technologies. PLA's lightweight nature facilitates the creation of quick and functional prototypes suitable for experimental evaluation.

However, for industrial applications, aluminium is a more suitable material for the robot's links. Aluminium's lightweight design allows for quick and precise movements while maintaining structural integrity.



Figure 3 Defining PLA material in Solidworks.

In the prototype phase of the project, the links were manufactured using PLA material and 3D printing. This method is cost-effective, allows for fast production, and helps test and improve the design. For industrial applications, aluminium and CNC machining will be used. Aluminium is lightweight, durable, and easy to machine, which improves the robot's efficiency. The design includes servo motors, fasteners, and standard gripper mechanisms. These components ensure the robot performs its movements accurately and efficiently.

E. Actuators and Additional Components

The manipulator system integrates four DS3225 servo motors as the primary actuators, driven by an ESP32 microcontroller for precise and reliable control. These servo motors provide smooth and accurate movement for the robotic arm's revolute joints, ensuring efficient operation across various tasks. The ESP32 offers flexibility in controlling the system and allows for potential expansion with additional sensors or peripherals.

To enhance the system's functionality and safety, additional components like cameras or other sensors can be integrated. For example, cameras can assist in object detection, path planning, or identifying obstacles in the workspace. Obstacle detection sensors, such as ultrasonic or infrared sensors, can further improve the manipulator's safety by preventing collisions during operation. These enhancements would provide the manipulator with advanced capabilities, creating a more intelligent and secure working environment.



Figure 4 DS3225 Servo Motor



Figure 5 ESP 32

III. DETAILED CALCULATIONS

A. Forward Kinematics

As the first step, we worked on the forward kinematics of the robot. To describe the robot, we used the Denavit-Hartenberg (DH) notation. The initial task was to assign coordinate frames for each joint to relate them to the world reference frame. These frames were prepared according to the DH notation, allowing us to mathematically represent the movements of the robot's joints.

Based on the defined reference frames, the link variables (link angle, twist angle, link length, and link offset) were identified, and the DH table was constructed. Using the DH table, homogeneous transformation matrices were derived, enabling transformations from any joint to the end-effector. These transformation matrices clearly define the forward kinematics of the system. The DH table for the designed robotic arm1 is provided below, and these transformation matrices clearly define the forward kinematics of the system.

TABLE I

D-H TABLE PARAMETERS FOR 4-DOF RRRR ROBOTIC ARM

Frame	Twist Angle (α)	Link Length	Offset (d)	Joint Angle (θ)
0 - 1	0	0	0	θ_1
1 - 2	-90	20mm	+45	θ_2
2 - 3	0	135mm	-45	θ_3
3 - 4	0	165mm	0	θ_4
4 - 5	0	80mm	0	0

Homogeneous Transformation Matrix For Joint 1:

$$\begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) & 0 & 0 \\ \sin(\theta_1) & \cos(\theta_1) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Transformation Matrix For Joint 2:

$$\begin{bmatrix} \cos(\theta_2) & -\cos(90)\sin(\theta_2) & -\sin(90)\sin(\theta_2) & 0 \\ \sin(\theta_2) & \cos(90)\cos(\theta_2) & \sin(90)\cos(\theta_2) & 0 \\ 0 & -\sin(90) & \cos(90) & 45 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Transformation Matrix For Joint 3:

$$\begin{bmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 & 135\cos(\theta_3) \\ \sin(\theta_3) & \cos(\theta_3) & 0 & 135\sin(\theta_3) \\ 0 & 0 & 1 & 45 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Transformation Matrix For Joint 4:

$$\begin{bmatrix} \cos(\theta_4) & -\sin(\theta_4) & 0 & 165\cos(\theta_4) \\ \sin(\theta_4) & \cos(\theta_4) & 0 & 165\sin(\theta_4) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 6 Transformation matrices for each joint.

B. Inverse Kinematics

To calculate the joint angle values in inverse kinematics, the second method, the geometric approach, was chosen over the

algebraic method. To reduce the complexity of the system, the first joint was analyzed separately from the second, third, and fourth joints. This allowed the first joint to be analyzed in two dimensions during the geometric analysis.

When the remaining second, third, and fourth joints were examined as a group, it was observed that their motion was confined to two-dimensional movement in the y-x plane. Consequently, the geometric analysis of this joint group was also performed in two dimensions, simplifying the derivation of the inverse kinematic equations significantly.

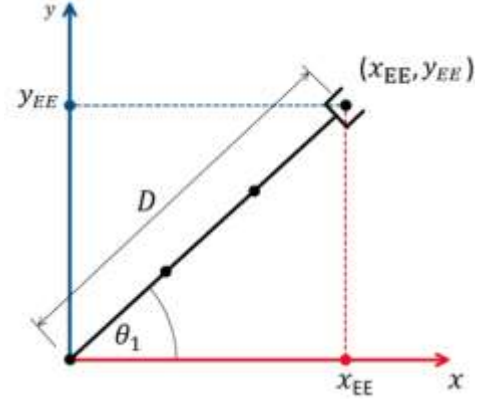


Figure 7 Geometrical analysis of the first joint.

The X-Y plane is considered the horizontal plane in this analysis. Movement within this horizontal plane is solely achieved by the first joint. The angular displacement of this joint is directly dependent on the X and Y coordinates. Therefore, when deriving the equation for the first joint, the angle is calculated as the arctan of the X and Y coordinates. Based on the representation shown in Figure 6, this approach ensures that the angular position of the first joint is accurately determined with respect to the horizontal plane.

$$\frac{y_{ee}}{x_{ee}} = \tan(\theta_1)$$

$$\theta_1 = \tan^{-1}\left(\frac{y_{ee}}{x_{ee}}\right)$$

When the remaining second, third, and fourth joints were examined, it was observed that their movement was confined to the Z-X plane. As a result, a two-dimensional analysis was conducted.

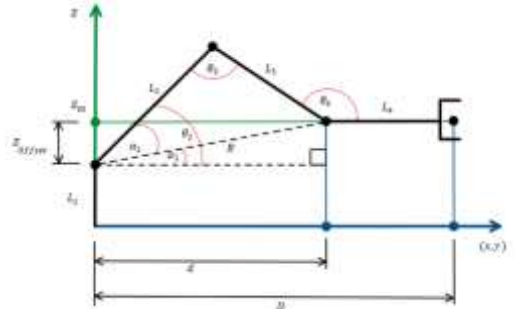


Figure 8 Geometrical analysis of the last three joints.

Based on the triangle formed by the second, third, and fourth joints shown in Figure 7, the following equations were derived. Using these equations, the formulas for the joint angles in inverse kinematics were obtained.

$$\begin{aligned}
 d &= D - L_4 \\
 Z_{offset} &= Z_{EE} - L_1 \\
 R &= \sqrt{(d)^2 + (Z_{offset})^2} \\
 \frac{d}{R} &= \cos(\alpha_1) \\
 \alpha_1 &= \cos^{-1}\left(\frac{d}{R}\right) \\
 (L_3)^2 &= (L_2)^2 + (R)^2 - 2 \times L_2 \times R \times \cos(\alpha_2) \\
 \alpha_2 &= \cos^{-1}\left(\frac{L_2^2 + R^2 - L_3^2}{2 \times L_2 \times R}\right) \\
 \theta_2 &= \alpha_1 + \alpha_2 \\
 R^2 &= (L_2)^2 + (L_3)^2 - 2 \times L_2 \times L_3 \times \cos(\theta_3) \\
 \theta_3 &= \cos^{-1}\left(\frac{L_2^2 + L_3^2 - R^2}{2 \times L_2 \times L_3}\right) \\
 \theta_4 &= 180^\circ - \{[180^\circ - (\alpha_2 + \theta_3)] - \alpha_1\}
 \end{aligned}$$

C. Forward and Inverse Dynamics

The torque calculation required for the system has been performed considering the force-torque relationship. These steps are explained sequentially below.

$$\tau_{ext} = J^T \cdot F_{ext}$$

Figure 9 Force- Torque relationship.

This equation relates the external torque F_{ext} applied on the robotic joints to the external force at the end-effector. The transpose of the Jacobian matrix acts as a mapping from the force applied in Cartesian space to the joint space. This is essential for including external effects, such as forces and torques, in the system dynamics.

1) Full Dynamics Equation

$$\tau_{(n \times 1)} = M(q)_{(n \times n)} \ddot{q}_{(n \times 1)} + C(q, \dot{q})_{(n \times n)} \dot{q} + G(q)_{(n \times 1)} + J_{(n \times 6)}^T F_{(6 \times 1)}$$

Figure 10 Torque formula.

This is the equation of motion for robotic systems. Each term represents a distinct dynamic contribution:

Where:

- n is the number of joints.
- $q(n \times 1)$ are the joint angles or distances.
- $\dot{q}(n \times 1)$ are the joint velocities.
- $\ddot{q}(n \times 1)$ are the joint accelerations.
- $\tau(n \times 1)$ is the joint torques or forces.
- $F(6 \times 1)$ is the Cartesian force or torque applied on the end-effector.
- $J^T (n \times 6)$ is the transpose of the Jacobian of the manipulator.
- $M(q)(n \times n)$ is the inertial matrix of the robot, depending on the configuration of the robot (joint angles).
- $C(q, \dot{q})(n \times n)$ is the Coriolis and centrifugal matrix, dependent on configuration and joint velocities.
- $G(q)(n \times 1)$ is the vector of gravity terms for each joint.

In this format, we only ignored the friction terms because they depend on the motor's coulomb viscous friction coefficients.

2) Operational Space Inertia Matrix

This equation transforms the joint space inertia matrix $M(\Theta)$ to operational (Cartesian) space. It ensures that the inertia matrix reflects the dynamics in the end-effector space, enabling better control and understanding of the forces and accelerations experienced in operational tasks.

$$M_x(\Theta) = J^{-T}(\Theta) M(\Theta) J^{-1}(\Theta),$$

$M_x(\Theta)$: Cartesian mass matrix

$J(\Theta)$: Jacobian matrix

$J^{-1}(\Theta)$: Inverse of the Jacobian matrix

$J^T(\Theta)$: Transpose of the inverse Jacobian matrix

$M(\Theta)$: Joint-space mass matrix

Θ : Joint-space position vector

3) Coriolis and Centrifugal Matrix

$$\begin{aligned}
 C &= \sum_{i=1}^n c_{ijk} \dot{q}_k \\
 c_{ijk} &= \frac{1}{2} \left[\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} + \frac{\partial M_{jk}}{\partial q_i} \right]
 \end{aligned}$$

These coefficients define the elements of the Coriolis matrix, capturing velocity-dependent effects. The Coriolis forces arise due to the motion of multiple robot links and the interaction between their velocities. The partial derivatives of the inertia matrix M with respect to joint positions determine these coefficients.

4) Gravity Terms

$$G_x(\Theta) = J^{-T}(\Theta)G(\Theta)$$

$$g_i = \sum_{j=1}^n m_j g_0^p J_j$$

This equation maps the gravity vector $G(\Theta)$ from joint space to operational (Cartesian) space. It enables accurate compensation for gravity effects on the end-effector, especially in tasks requiring precise positioning or manipulation. Also it calculates the gravitational torque g_i acting on joint i . The gravitational contribution of each link j depends on its mass m_j the gravitational acceleration g_0^p in the base frame, and the Jacobian J_j . Summing these contributions provides the total gravitational torque on each joint.

IV. ELECTRONIC UNIT DESIGN

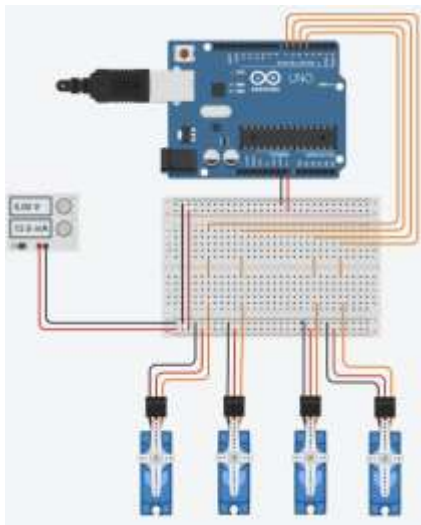


Figure 11 Electronic Control Unit Design.

In this project, the electronic control unit for the robotic arm is designed using an ESP32 microcontroller, a breadboard, and four DS3225 servo motors. The system is powered by an external 5V power supply to ensure stable operation for all servo motors. The ESP32 acts as the main controller, sending PWM signals to the servo motors to control their movement. The positive and ground terminals of the external power supply are connected to the breadboard, distributing power to the servo motors. Each servo motor is connected to the breadboard, where the red wire (positive) is connected to the positive rail, the brown wire (ground) is connected to the ground rail, and the orange wire (signal) is connected to a PWM output pin on the ESP32. This configuration enables precise control of the robotic arm's joints while ensuring the motors receive sufficient power for optimal performance.

IV. CODING THE MICROCONTROLLER

To control end effector positions in the microcontroller, a servo motor control algorithm is used. Each servo motor is assigned a target position, and an interpolation function is used to gradually reach that target. The end effector's positions are determined based on user inputs or a predefined trajectory plan.

The main steps that our microcontroller have are:

- Define the initial positions.
- Set the target positions.
- Gradually move each servo from the current position to the target position in small steps.
- Once all servos reach their positions, stabilize the system and continuously monitor for external changes.

```
Start
Define servos
Move to initial positions

Infinite loop:
  Read target positions
  For each servo:
    If current position ≠ target position:
      Take a small step
      Update current position
    If current position = target position:
      Maintain position
  End loop
```

Figure 12 Pseudocode of Servo Motor Controllers.

The loop frequency is usually set to 50 Hz (20 ms loop time). We choose this frequency because servo motors usually operate on a 50 Hz PWM signal and provide smooth movements at this frequency. The loop is fast enough to provide timely feedback without overloading the microcontroller.

V. MATLAB – SIMULINK SIMULATION

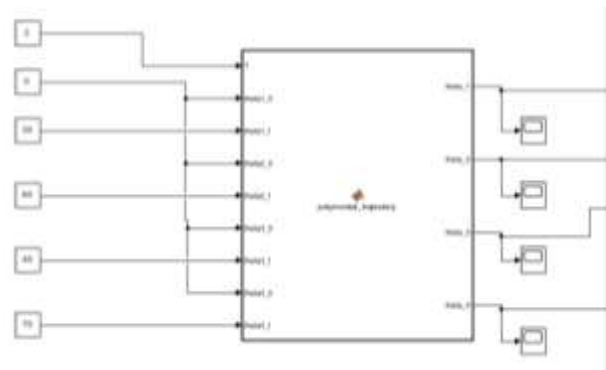


Figure 13 Polynomial Trajectory

We use polynomial equations (5th degree) to calculate the joint angles of a robot manipulator. The input angles are processed to create a smooth trajectory for each joint based on predefined start and end positions, providing smooth motion. This system provides the angular positions of the robot arm joints as a function of time, which is critical for simulating realistic and smooth motions.

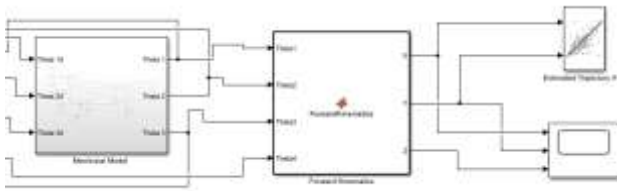


Figure 14 Mechanical Model and Forward Kinematics.

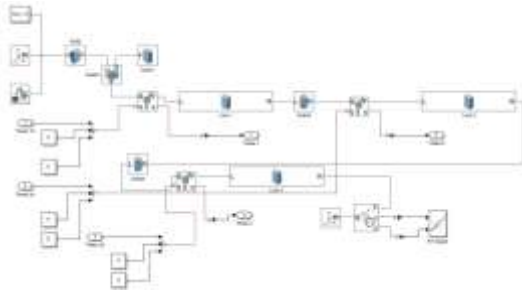


Figure 15 Mechanical Model Detail.

We designed the mechanical model in Simulink using block manipulation, since CAD designs cannot be imported directly. Blocks represent the physical structure of the robot, including links and joints. We implemented forward kinematics using transformation matrices. This allows me to calculate the X, Y, and Z coordinates of the end effector based on the joint angles. The forward kinematics block determines the position of the robotic arm in the workspace, allowing its motion to be accurately simulated.

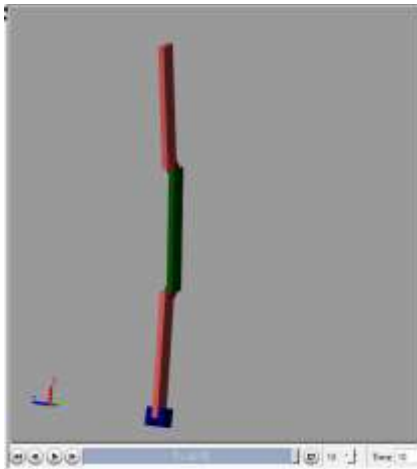


Figure 16 Simulation of Robotic Arm in Simulink.

The simulation visually shows the movement of the robotic arm according to the given joint angles and PID controller values. The 3D environment allows me to observe how the arm moves and how the control parameters affect its trajectory.

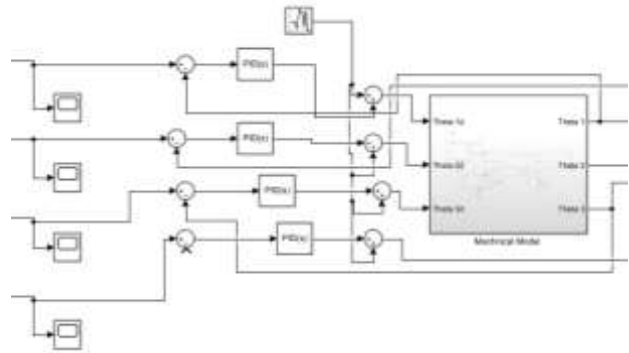


Figure 17 PID Implementation.

We implemented PID controllers to dynamically adjust the joint angles of the robotic arm.

Advantages of PID Controllers:

- They provide smooth and stable motion by minimizing the errors in the joint angles.
- They improve the response time of the system and reduce oscillations.

Also, we added the disturbance to test the robustness of the system. The PID controller adjusts the system in real time to compensate for the distortions and ensures that the arm remains on the desired path.

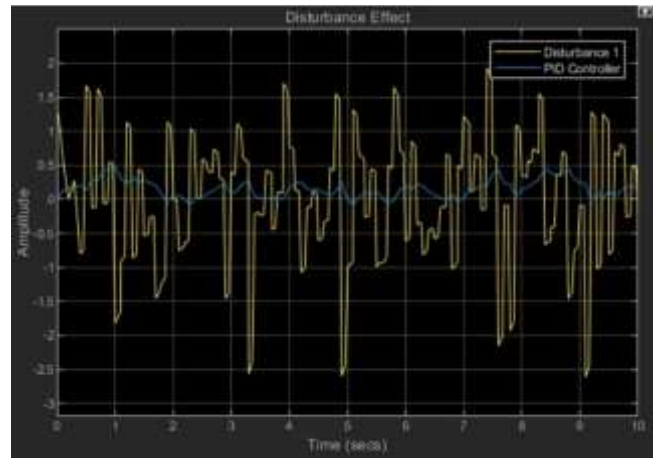


Figure 18 Disturbance effects and PID.

VI. APPLYING TRAJECTORY

The trajectory is obtained by using fifth-order polynomial equations as shown in the provided formula. These equations are used to calculate the trajectory coefficients (a_0 to a_5) for each joint. The coefficients are derived from boundary conditions including the starting and ending positions, velocities, and accelerations. By including these conditions, we ensure a smooth transition between the starting and ending points of the motion while maintaining continuous position, velocity, and acceleration.

$$\begin{aligned}
a_0 &= \theta_0, \\
a_1 &= \dot{\theta}_0, \\
a_2 &= \frac{\ddot{\theta}_0}{2}, \\
a_3 &= \frac{20\theta_f - 20\theta_0 - (8\dot{\theta}_f + 12\dot{\theta}_0)t_f - (3\ddot{\theta}_0 - \ddot{\theta}_f)t_f^2}{2t_f^3}, \\
a_4 &= \frac{30\theta_0 - 30\theta_f + (14\dot{\theta}_f + 16\dot{\theta}_0)t_f + (3\ddot{\theta}_0 - 2\ddot{\theta}_f)t_f^2}{2t_f^4}, \\
a_5 &= \frac{12\theta_f - 12\theta_0 - (6\dot{\theta}_f + 6\dot{\theta}_0)t_f - (\ddot{\theta}_0 - \ddot{\theta}_f)t_f^2}{2t_f^5}.
\end{aligned}$$

Figure 19 Coefficient calculation for trajectory equations.

The code takes as input the values θ_0 , θ_f , and T (total time). Using these, the polynomial coefficients are calculated directly from the equations. These coefficients (a_0 to a_5) are then applied to calculate the position, velocity, and acceleration at each time step. This approach ensures that the robotic arm moves smoothly, avoiding sudden changes in motion.

By integrating these equations into the code, we aim for the closest trajectory to the optimum against the physical constraints of the robot, such as avoiding sudden jolts or stresses in mechanical components. This method makes the movement safe, efficient, and realistic for the task. The results are plotted for each joint to visualize the trajectories, and user-defined inputs significantly affect the result, such as the range and duration of the movement.

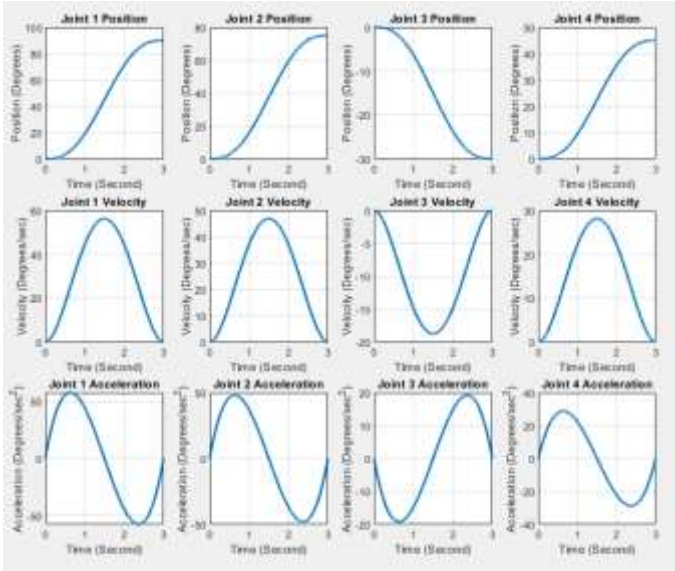


Figure 20 Trajectory results graph for each joint.

VIII. REAL TIME APPLICATION

We built a real-life robotic arm. The robot's design includes precise motion control and stability. While the end-effector

mechanism has been designed and integrated, it has not been fully utilized in the current stage of implementation.



Figure 21 4-DOF real robot implementation.

VII. CONCLUSION

In conclusion, this project has successfully demonstrated the design and implementation of a 4-DOF robotic arm, integrating precise kinematic modeling with efficient electronic control. The forward kinematics of the system were modeled using Denavit-Hartenberg (DH) parameters, allowing the end-effector's position and orientation to be accurately calculated through transformation matrices. Inverse kinematics were solved using a geometric approach, simplifying the complexity of the calculations by analyzing joint groups in two-dimensional planes. This provided a streamlined and effective method for determining joint angles to achieve desired end-effector positions.

The electronic control unit, built with an ESP32 microcontroller and DS3225 servo motors, seamlessly translates these kinematic solutions into precise motion. The use of a 5V power supply ensures reliable operation of the motors, while the modular nature of the design allows for future upgrades, such as integrating sensors for obstacle detection or advanced feedback systems. This combination of advanced kinematic modeling and robust electronic control highlights the potential of the robotic arm to perform complex tasks with accuracy and efficiency, laying a strong foundation for future enhancements and applications in industrial or academic settings.

REFERENCES

- [1] P. Shahverdi and M. T. Masouleh, "A Simple and Fast Geometric Kinematic Solution for Imitation of Human Arms by a NAO Humanoid Robot," in Proceedings of the 4th International Conference on Robotics and Mechatronics, Tehran, Iran, Oct. 26-28, 2016, pp. 572-577,

- [2] J. J. Craig, Introduction to Robotics: Mechanics and Control, 3rd ed. Harlow, Essex, England: Pearson Education, 2014, pp. 1–360.
- [3] M. Kaur, S. Sondhi, and V. K. Yanumula, “Kinematics Analysis and Jacobian Calculation for Six Degrees of Freedom Robotic Arm,” in Proceedings of the 17th India Council International Conference (INDICON), New Delhi, India, Dec. 2020, pp. 1-6, doi: 10.1109/INDICON49873.2020.9342093.